SESSION 8

Software Design Principles

AI Career Guidance & Skill Recommendation System

Project	AI Career Guidance & Skill Recommendation System
Deliverable	Design Analysis Report
Industry Mapping	Software Design Engineering
Prepared by	Manya Goyal, B.Tech, KIET Group of Institutions
Team Members	Pawni Agrahari
Repository	[Add GitHub repository link]

Topics Covered in This Session

- Modularization
- Coupling and cohesion analysis
- Function-oriented vs object-oriented design
- Top-down and bottom-up design

1. Introduction

Good design makes software easy to understand, change, test and reuse. This report analyses the design of the AI Career Guidance & Skill Recommendation System against four fundamental principles: modularization, coupling, cohesion and the choice of design approach.

2. Modularization

The system is divided into modules, each owning one area of responsibility and exposing a small interface. Modules are mapped to folders in the repository.

Module	Responsibility	Public interface (examples)	Depends on
auth	Registration, login, JWT, role checks	register(), login(), verifyToken()	User model
resume	Upload, text extraction, AI parsing	uploadResume(), parseResume()	aiService, Resume model
aiService	Only place that calls Claude API	analyzeResume(), generateRoadmap(), answerQuery()	Claude API
career	Career and skill catalogue, matching logic	getCareers(), computeMatch()	Career, Skill models
gap	Skill gap calculation	computeGap(profile, career)	career
roadmap	Roadmap creation and progress	createRoadmap(), updateStep()	gap, aiService
course	Course catalogue and suggestions	coursesForSkill()	Course model
chat	Chat sessions and context building	sendMessage(), getHistory()	aiService, resume
analytics	Aggregated statistics (Python + Pandas)	topSkills(), commonGaps()	DB read access
admin	CRUD for careers, courses, users	create/update/delete	career, course, auth

Benefits achieved: team members work on different modules in parallel; AI provider changes affect only aiService; each module can be unit tested alone.

3. Coupling Analysis

Coupling measures how strongly one module depends on another. Lower coupling is better. Levels from worst to best: content, common, control, stamp, data coupling.

Module pair	Coupling type	Level	Analysis / improvement
resume to aiService	Data	Low (good)	Passes only rawText and returns JSON; provider can change freely.
career to gap	Data	Low (good)	Passes profile skills and career requirements as plain data.
roadmap to gap	Stamp	Medium	Receives the whole gap report object; pass only missing skill list to reduce coupling.
chat to resume	Stamp	Medium	Reads full parsed profile; use a small profileSummary() function instead.
controllers to models	Data via Mongoose	Medium	Direct model use in controllers; move queries into a service/repository layer.
Any module to env config	Common (global)	Medium	Read config once in a config module and inject values.

Outcome: no content coupling and no control coupling (flags that tell another module what to do) is allowed. Remaining stamp and common coupling is reduced during implementation.

4. Cohesion Analysis

Cohesion measures how closely the elements inside one module belong together. Higher cohesion is better. Levels from worst to best: coincidental, logical, temporal, procedural, communicational, sequential, functional.

Module	Cohesion type	Rating	Reason
career.computeMatch()	Functional	Best	Single task: compute weighted match score.
aiService	Functional	Best	Everything relates to talking to the AI provider.
resume pipeline	Sequential	Good	Validate, extract text, parse: output of one step feeds the next.
gap module	Communicational	Good	Functions work on the same profile and career data.
utils.js (initial draft)	Logical / coincidental	Poor	Mixed helpers; split into dateUtils, skillUtils, fileUtils.
admin controller	Logical	Fair	Groups CRUD for different entities; split by entity.

Goal: high cohesion and low coupling. Where ratings are Fair or Poor, the improvement is recorded in the backlog.

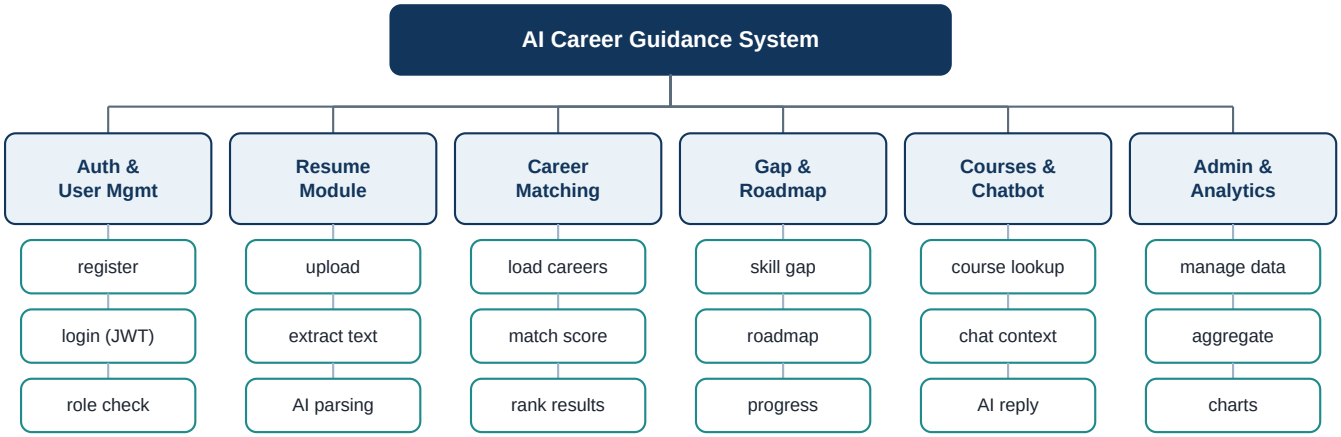
5. Function-Oriented vs Object-Oriented Design

Aspect	Function-oriented design	Object-oriented design
Main idea	System is a set of functions that transform data (data flow diagrams)	System is a set of objects combining data and behaviour (class diagrams)
Data handling	Shared data passed between functions	Data hidden inside objects (encapsulation)
Reuse	Reuse of functions	Reuse through inheritance and composition
Change impact	Data structure change may affect many functions	Changes localised to a class
Best for	Pipelines, calculations, batch processing	Entity-rich systems with many related concepts
In this project	Match score, gap calculation, Pandas analytics, resume pipeline	User, Student, Resume, Career, Roadmap, Course models and services

Decision: A **hybrid** design is used. The domain is modelled with object-oriented classes (Session 6), while calculations such as match scoring and analytics are written as pure functions that are easy to test. JavaScript and Mongoose support both styles naturally.

6. Top-Down Design

Top-down design starts from the whole system and repeatedly breaks it into smaller parts until each part is simple enough to code.



Level 0: system | Level 1: modules | Level 2: functions

Figure 1: Top-down functional decomposition

7. Bottom-Up Design

Bottom-up design starts from small reusable building blocks and combines them into larger units. Used for utilities and infrastructure in this project:

1. **Utilities:** normalizeSkill(), parseJSONSafe(), fileValidator(), weightedScore().
2. **Infrastructure:** database connection, auth middleware, Claude client with retry and timeout.
3. **Services:** resumeService, matchService, gapService, roadmapService built from utilities.
4. **Controllers and routes:** combine services into API endpoints.
5. **UI:** reusable React components (SkillChip, ScoreCard, RoadmapStep) assembled into pages.

Approach	Where used	Advantage	Risk
Top-down	Overall module structure, API design	Clear big picture, early requirement coverage	May miss reuse opportunities
Bottom-up	Utilities, shared components, AI client	Reuse and early testing of building blocks	Pieces may not fit the whole

Conclusion: The team designs top-down to define modules and interfaces, and builds bottom-up to create reusable parts first.